

闭环优于一次成型：FJSP+MAPF 耦合系统的 在线重调度编排与承诺一致性修订

SkyResearch

2026 年 9 月 1 日 v1 草稿

摘要

在 FJSP 与 MAPF 耦合的车间系统中 (I-FJSP-MAPF, 见姊妹篇规约), 现有最先进做法 (如 Li 等 2026 的 JSP+MAPF 框架) 采用 “一次规划—执行期修正” 的准开环范式。本文 (1) 将耦合问题形式化为**在线版本** O-IFJSP-MAPF: 外生扰动过程、状态轨迹、闭环策略空间与扰动遗憾 (disruption regret); (2) 定义**承诺一致性修订**语义——任何在线重规划必须与已执行前缀一致且不违背已冻结承诺, 该语义与引擎的 PlanRevision/激活机制一一对应; (3) 提出**恢复编排器** (Recovery Orchestrator): 以 (τ, κ) 策略空间补上引擎缺失的闭环策略层; (4) 40 个扰动 episode 的试点给出三个出人意料的结论: 一次成型 CP-SAT 计划对 200 步单机停机仅退化 4–14% (“等待—吸收” 的被动鲁棒性); 事件触发的全域修订在在制扰动下全部激活失败 (未解决承诺) 而局部修复多为空作用域——**闭环的瓶颈在承诺解除机制而非触发策略**; 失败的重规划尝试本身有 7% 的负作用。该负结果把闭环协同的研究议程从策略层重定向到机制层 (可违约承诺语义)。

1 引言

姊妹篇(方向 2)定义了 I-FJSP-MAPF 的静态规约与分层执行语义 $\mathcal{H} = (\sigma_J, \sigma_A, \sigma_R)$ 。本篇处理其在线版本: 执行期内发生外生扰动 ξ (机器故障与修复、AGV 故障、临时路障) 时, 谁在何时以多大范围重调度?

现状的三个断层:

1. **引擎断层**: SkyEngine (及同类系统) 的调度层已暴露修订 API (残差问题构建、承诺冻结、激活与回滚), 但不存在调用它们的策略组件——扰动下系统实际上是开环的;
2. **文献断层**: 动态 FJSP 工作考虑机器/AGV 故障但不含栅格冲突路由; MAPF 的滚动方法 (RHCR [1]) 考虑执行偏差但无调度层; Li 等 [2] 自述其框架为一次性 “generate-then-refine”, 闭环列为未来工作;

3. **评测断层**：无同时度量扰动遗憾与重调度代价（修订次数、求解时间、计划紧张度）的协议。

本文贡献：O-IFJSP-MAPF 在线规约 (§2)、承诺一致性修订语义 (§3)、恢复编排器设计与策略空间 (§4)、四策略扰动试点 (§5)。

2 在线问题规约 O-IFJSP-MAPF

2.1 外生扰动过程

在静态输入（姊妹篇 §3.1）之上，定义扰动过程 $\xi = \{\xi_t\}_{t \geq 0}$ ， ξ_t 为 t 时刻发生的事件集合，取值于：

$$\xi_t \in \{\mathbf{m_down}(m, d), \mathbf{m_up}(m), \mathbf{a_down}(k, d), \mathbf{a_up}(k), \mathbf{obs}(v, d)\} \quad (1)$$

分别表示机器 m 故障 d 步/修复、AGV k 故障/修复、单元 v 临时封闭 d 步。扰动可为预定表 (schedule) 或随机过程 (MTBF/泊松到达)，由外生种子独立于决策种子控制 (可复现性)。

2.2 状态、策略与目标

定义 1 (在线耦合调度策略). 策略 π 在每步 t 观测状态 $s_t = (\text{剩余工序}, \text{机器/AGV 状态}, \text{AGV 位置}, \dots)$ 输出 (i) 是否触发修订及修订范围 (本篇的编排决策), (ii) 运输任务分派, (iii) AGV 动作。轨迹由分层求解器 $\mathcal{H}$ 与环境演化共同产生。

定义 2 (扰动遗憾). 固定扰动场景 S (含空场景 $S_0 = \emptyset$)，定义

$$\text{Regret}(\pi; S) = C_{\max}(\pi, S) - C_{\max}(\pi, S_0), \quad \text{RRegret}(\pi; S) = \frac{C_{\max}(\pi, S)}{C_{\max}(\pi, S_0)}.$$

目标：在场景分布 $\mathcal{D}$ 上最小化 $\mathbb{E}_{S \sim \mathcal{D}}[\text{RRegret}]$ ，同时以修订次数 N_{rev} 与编排求解时间 T_{plan} 为约束/次级目标。（竞争比式分析留作后续理论工作。）

3 承诺一致性修订

在线重规划不能从零开始：执行已发生、承诺已发出。

定义 3 (可行修订). $\Pi_{t+1} = \rho(\Pi_t \mid s_t)$ 为可行修订，若满足：

1. **执行一致** (*weak execution consistency*): Π_{t+1} 在 t 时刻前的调度/路径与已执行轨迹逐点一致；
2. **承诺一致** (*commitment consistency*): 已开工工序不被迁移、在途运输任务 (AGV 已取货) 必须按原目的地交付、已冻结指派不变；

3. **残差最优**: 在 1-2 约束下, Π_{t+1} 为残差问题 (剩余工序 + 未冻结决策) 上的 (近似) 最优。

命题 1 (锚定下界不变性). 若修订保持承诺一致性, 则任意时刻的已完成 + 在途部分构成最终 $C_{\max}$ 的单调不减下界; 因此 $\text{Regret} \geq 0$ 且仅由未冻结部分的次优性与等待代价构成。

(证明平凡, 从承诺一致性直接得出; 意义在于把遗憾归因拆分为“冻结代价 + 残差再优化不足”两部分, 分别对应 $\kappa = \text{partial}$ 冻结过多与 $\kappa = \text{full}$ 求解限时不足两种失效模式。)

与引擎的对应。残差问题构建 = `build_residual_problem`; 承诺分类 (transport/processing 冻结) = `frozen_operations`; 激活校验 = `execution_ready` 与激活失败回滚; 修订账本与紧张度 = `PlanRevisionLedger` 与 `ScheduleRevisionTracker`。本篇不改动这些机制, 只补策略层。

4 恢复编排器

Algorithm 1 Recovery Orchestrator (每步调用)

```

1:  $E_t \leftarrow \text{injector.get\_step\_events}()$ 
2: if  $\tau = \text{never}$  或  $\sigma_J$  为规则求解器 then
3:   return
4: end if
5:  $e \leftarrow \text{TRIGGER}(\tau, E_t, t; \Delta_{\min})$  ▷ 含最小重规划间隔  $\Delta_{\min}$  防风暴
6: if  $e = \perp$  then
7:   return
8: end if
9: if  $\kappa = \text{partial}$  and  $e$  为机器故障 then
10:   $\rho \leftarrow \text{request\_partial\_schedule\_repair}$  (围绕故障机器的影响域)
11: else if  $\kappa = \text{full}$  then
12:   $\rho \leftarrow \text{request\_plan\_revision}$  (全域残差, 承诺冻结, 激活)
13: end if
14: 激活失败  $\Rightarrow$  记账回滚 (开环降级一步)
```

策略空间与对照:

- **greedy-reactive**: 规则调度每步重建 (规则级闭环, 无修订账本);
- **cpsat-static**: $\tau = \text{never}$ (一次成型开环对照);

- **cpsat-full**: $\tau = \text{event}$, $\kappa = \text{full}$;
- **cpsat-partial**: $\tau = \text{event}$, $\kappa = \text{partial}$ 。

假设: H1 cpsat-full/partial 的 RRegret 显著低于 cpsat-static (闭环价值); H2 扰动强度增大时 partial 的求解开销优势扩大 (范围效应); H3 static 在无扰动场景最优 (开环无修订税)。

5 试点实验

设置: $\{\text{mk01}, \text{mk02}\} \times \text{迷宫地图} \times 4 \text{ AGV} \times \text{场景} \{S_0, S_1=\text{m_down}@150 \times 40, S_2=S_1+\text{a_down}@300} \times 4 \text{ 策略}; \text{greedy 系用内置 A* 路由, CP-SAT 系用滚动 EECBS}; \text{episode 上限 4096 步, 进程级硬超时}。$

实例	策略	S_0	S_1	S_2	S_3	$S_4(\text{强})$
mk01	greedy-reactive	1099	1099	1155	977	595
mk01	cpsat-static	408	408	423	449	467
mk01	cpsat-full	408	408	423	449	467
mk01	cpsat-partial	408	408	423	449	467
mk02	greedy-reactive	4096 [†]	3417	781	1103	1132
mk02	cpsat-static	410	410	426	427	426
mk02	cpsat-full	410	410	426	427	457 [‡]
mk02	cpsat-partial	410	410	426	427	426

表 1: 闭环试点 makespan (步)。†: 饥饿型活性失稳 (姊妹篇发现 6, 4096 步未完工); ‡: 失败修订尝试的规划延迟副作用。三个 CP-SAT 策略几乎处处同分——见发现 F2。

发现:

1. **F1 (H3 成立)**: 无扰动时名义开环 CP-SAT 大幅最优 (408 vs 1099; mk02 上 greedy 甚至失稳); 即使 S_4 的 200 步单机停机也只使静态计划退化 4-14%——运输请求的“按计划 ready 时刻释放 + 机器等待”语义构成了被动鲁棒性;
2. **F2 (H1 在当前栈被拒绝——机制瓶颈)**: 事件触发的全域修订 (U4) 在在制扰动下全部激活失败 (“未解决承诺”: 被打断的在制工序使残差问题不可满足承诺约束), 局部修复 (U3) 多数返回空作用域 (计划松弛吸收扰动)——因此 full/partial/static 三臂不可区分。闭环的瓶颈不在触发策略 (τ, κ) , 而在承诺解除机制: 需要 “可违约 + 罚金” 的承诺语义 (引擎侧改造项), 这把议程从策略层重定向到机制层;

3. **F3**: greedy-reactive 对扰动的增量很小 (mk01: $1099 \rightarrow 1155$; S_4 下甚至 $595 < 1099$ ——停机期间负载下降), 但其绝对水平始终劣于 CP-SAT 系 44–72%, 且存在失稳风险——“反应式 = 鲁棒”的直觉只对增量成立;
4. **F4**: 失败的重规划尝试有真实代价 (mk02 S_4 : full 457 vs static 426, 规划延迟挤占执行) ——重规划要么激活成功, 否则不如不做, 强化了 F2 的机制先行结论。

6 正式实验计划（待评估后执行）

(i) 引擎侧: 可违约承诺语义 (罚金参数化) + 机器恢复事件的修订窗口; (ii) 实验侧: 实例扩至 mk01–mk10 $\times$ 2 地图 $\times K \in \{2, 4, 6\} \times$ 场景 $\{S_0..S_4, \text{stress}\} \times 4$ 策略 $\times 3$ 种子 ≈ 1680 episode, 约 7–10 小时分批; (iii) $\tau = \text{periodic-}K$ 扫描与 $\Delta_{\min}$ 消融。
[FULL_RUN_PLACEHOLDER]

7 结论

本篇把 FJSP+MAPF 耦合系统从“一次规划”推进到“闭环编排”: 给出在线规约、与引擎修订机制一一对应的承诺一致性语义, 以及 (τ, κ) 恢复编排器。试点量化闭环相对开环的扰动遗憾收益与修订税的权衡。代码见 sky_research 仓库 0901closeloop 分支。

参考文献

- [1] Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W. Durham, T. K. Satish Kumar, and Sven Koenig. Lifelong multi-agent path finding in large-scale warehouses. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.
- [2] Rui Li et al. A new framework for job shop integrated scheduling and vehicle path planning problem. In *Sensors*, volume 26, 2026.